

PointProgrammerNet: Fixed-state additive programs for streaming point-cloud understanding

Dian Yu^{a,**}

^aUniversity of New South Wales, Sydney, NSW 2052, Australia

ABSTRACT

Point-cloud networks often recover context by revisiting points, selecting neighborhoods, or applying a global maximum. These operations are effective offline but make a growing stream hard to summarize without retaining or reprocessing its history. PointProgrammerNet instead writes observations into fixed-size fast-weight-programmer (FWP) matrices. Each write is a continuous normalized outer product, so chunks can be added, merged, removed, or decayed. The final models retain three 128×170 float32 states (255 KiB), independent of point count, and never apply a maximum over decoded points. A coordinate-conditioned decoder segments arbitrary query locations; a state-only residual decoder classifies after all points are discarded. We prove permutation invariance, chunk equivalence, fixed retained memory, and continuous evidence paths. With XYZ input, the model obtains 92.90% point accuracy, 82.39% instance mIoU, and 78.68% category mIoU on ShapeNetPart, and 86.59% ModelNet40 accuracy without retaining a point list. At 8,192 accumulated points, incremental state construction is $5.54\times$ faster than rebuilding, with relative state error below 8×10^{-7} . The model offers a measured accuracy–streamability tradeoff rather than replacing hierarchical offline networks.

© 2026 Elsevier Ltd. All rights reserved.

Keywords: point clouds ; streaming learning ; additive state ; fast-weight programmers ; segmentation ; classification

1. Introduction

A point cloud is an unordered set, but a sensor produces it over time. Within a frame, processing should be permutation invariant; across frames, it should avoid revisiting all history. PointNet Qi et al. (2017a) uses a symmetric maximum, while PointNet++ Qi et al. (2017b) builds multiscale metric neighborhoods. A global maximum is not an additive state: deletion, independent chunk merging, and continuously weighted evidence cannot generally be expressed by adding fixed summaries.

We impose a stricter constraint: after a point is written, it may be discarded. Later computation accesses only fixed matrices, plus a requested coordinate for segmentation. A learned key distributes each point across spatial rows and a second-order value records mass, position, learned content,

and cross moments. The resulting state is a sum of per-point programs.

This permits independent chunks to merge, new points to update without a pass over history, stored subsets to be subtracted, and old evidence to decay. Segmentation reads a coordinate-conditioned field and stores no historical point features. Classification reads only fixed rows and deletes the complete point list.

Our contributions are:

- a three-scale second-order FWP state with fixed retained memory and unordered additive writes;
- task-dual no-global-max readouts: an information-preserving coordinate bypass plus independent FWP queries for segmentation, and state-only residual classification;
- proofs of merge equivalence, fixed memory, and continuous evidence and query paths;

^{**}Corresponding author
e-mail: dian.yu2@student.unsw.edu.au (Dian Yu)

- three-seed accuracy, algebra, architecture, and timing studies, including 5.54× speedup at 8,192 points.

Our local PointNet++ implementations are controlled references, not canonical reproductions. The main claim is the update behavior and its measured accuracy cost.

2. Related Work

PointNet applies pointwise transforms and maximum pooling Qi et al. (2017a); PointNet++ adds hierarchical local metric structure Qi et al. (2017b). Deep Sets studies invariant sum-based functions Zaheer et al. (2017), and Set Transformer uses attention and inducing points Lee et al. (2019). Kernelized attention admits recurrent computation Katharopoulos et al. (2020); additive outer-product memories and delta corrections connect it to fast-weight programmers Schlag et al. (2021). Our contribution is their geometric, second-order use in a point-cloud architecture that discards historical points. We evaluate ModelNet40 Wu et al. (2015) and ShapeNetPart Yi et al. (2016), derived from ShapeNet Chang et al. (2015).

3. Method

Let $X = \{\mathbf{x}_i\}_{i=1}^N$, $\mathbf{x}_i \in \mathbb{R}^3$. Reported models use XYZ; normals or other attributes may extend values without changing the algebra. We require

$$\mathbf{S}(A \uplus B) = \mathbf{S}(A) + \mathbf{S}(B). \quad (1)$$

Three fine, middle, and broad branches each use $R = 128$ rows and $D = 170$ columns. Their learnable radii start at 0.08, 0.20, 0.60. Figure 1 summarizes the shared additive encoder and the two task-specific readouts.

3.1. Continuous keys and second-order writes

Branch s has Sobol-initialized trainable centers $\mathbf{c}_{sj}$ and inverse bandwidth γ_s :

$$\ell_{sj}(\mathbf{x}) = -0.5\|\gamma_s(\mathbf{x} - \mathbf{c}_{sj})\|_2^2, \quad (2)$$

$$k_{sj}(\mathbf{x}) = \exp \ell_{sj}(\mathbf{x}) / \sum_{r=1}^R \exp \ell_{sr}(\mathbf{x}). \quad (3)$$

Thus every point continuously distributes unit mass across rows, without hard assignment or point comparison. A branch MLP gives $\mathbf{g}_s(\mathbf{x}) \in \mathbb{R}^{32}$ and

$$\mathbf{v}_s(\mathbf{x}) = [1, \mathbf{x}, \mathbf{g}_s, \text{vec}(\mathbf{x}\mathbf{g}_s^\top), x^2, y^2, z^2, xy, xz, yz, \mathbf{g}_s \odot \mathbf{g}_s] \in \mathbb{R}^{170}. \quad (4)$$

The FWP is

$$\mathbf{S}_s(X) = \sum_i \mathbf{k}_s(\mathbf{x}_i) \mathbf{v}_s(\mathbf{x}_i)^\top = \mathbf{K}_s^\top \mathbf{V}_s \in \mathbb{R}^{128 \times 170}. \quad (5)$$

Mass permits normalized means; other columns yield centroids, feature means and deviations, centered spatial moments, and position–feature cross moments.

3.2. Segmentation readout

At query $\mathbf{q}$, $\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s$. Density-normalized, query-centered moments pass through a branch MLP to $\mathbf{r}_s(\mathbf{q}) \in \mathbb{R}^{160}$. The head is

$$\mathbf{z}_{\text{seg}}(\mathbf{q}) = h_{\text{seg}}([\mathbf{r}_m; \mathbf{r}_f; \mathbf{r}_b; \phi(\mathbf{q}); \mathbf{e}_{\text{cat}}]). \quad (6)$$

Here $\phi(\mathbf{q}) \in \mathbb{R}^{32}$ is a clean coordinate feature and $\mathbf{e}_{\text{cat}}$ is the ShapeNetPart category. The route $\mathbf{q} \mapsto \phi(\mathbf{q}) \mapsto h_{\text{seg}}$ bypasses FWP compression. It is therefore a U-Net-style skip in function: the FWP branches supply compressed multiscale context, while the bypass preserves the exact identity and location of the queried point. It is not a conventional U-Net skip between matched-resolution feature maps, so we call it a coordinate skip. Reads and the coordinate skip are concatenated directly; query points never interact and no decoded-point maximum is used. After writing, observed features may be deleted. Only coordinates of interest need be retained. Any coordinate can be queried, defining an implicit state-conditioned field. We do not yet evaluate accuracy at unobserved or off-surface queries.

3.3. Classification readout

Each state row is decoded into coordinate-free statistics, tokenized, and summarized by two soft row weightings. This processes 128 fixed rows, not input points. It yields $\mathbf{h}_f, \mathbf{h}_m, \mathbf{h}_b \in \mathbb{R}^{256}$:

$$\boldsymbol{\delta} = \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \quad (7)$$

$$\mathbf{z}_{\text{cls}} = h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \boldsymbol{\delta})). \quad (8)$$

The zero-initialized bounded residual makes the middle branch a stable reference. Points are deleted before this head. Only the middle radius uses a 25× learning-rate multiplier.

3.4. Formal properties

Proposition 1 (Permutation invariance and additivity). For each branch, equation (5) is invariant to point order and $\mathbf{S}_s(\uplus_t X_t) = \sum_t \mathbf{S}_s(X_t)$.

Proof. Each point contributes one matrix. A finite sum is invariant to reordering and regrouping in real arithmetic; float differences arise only from accumulation order. $\square$

Corollary 1. Chunks merge by addition. If subset state $\mathbf{S}_s(Y)$ is available, $\mathbf{S}_s(X \setminus Y) = \mathbf{S}_s(X) - \mathbf{S}_s(Y)$. Exponential forgetting uses $\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t)$, $0 \leq \rho \leq 1$.

Proposition 2 (Fixed memory and update). Retained memory is $O(\sum_s R_s D_s)$, independent of observed-point count. Encoding M new points costs $O(M \sum_s R_s D_s)$ for the outer products and is independent of history length.

Proof. Only fixed matrices persist; a new chunk produces the same matrix products regardless of earlier chunks. $\square$

Proposition 3 (Continuous evidence and queries). For evidence strength $t \in [0, 1]$, let

$$\mathbf{S}_s(t) = \mathbf{S}_s^{\text{old}} + t \mathbf{k}_s(\mathbf{x}) \mathbf{v}_s(\mathbf{x})^\top. \quad (9)$$

Classification logits and fixed-query segmentation logits are continuous, piecewise-smooth in t . Fixed-state segmentation logits are continuous, piecewise-smooth in $\mathbf{q}$.

Proof. The state is affine in t . RBFs, softmax, affine maps, positive-floor normalization, ReLU/GELU, and stabilized square roots are continuous; their composition is continuous and piecewise differentiable. $\square$

The state changes linearly with evidence strength. Neural logits and probabilities generally change nonlinearly but continuously; a hard argmax label may switch at a boundary. The property needs no empirical proof, though calibration and temporal stability do.

3.5. Future frame-level delta rule

Let $\mathbf{K}_{s,t}, \mathbf{V}_{s,t}$ package all points of frame t . A future adaptive transition is

$$\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \eta_t \mathbf{K}_{s,t}^\top (\mathbf{V}_{s,t} - \mathbf{K}_{s,t} \mathbf{S}_{s,t-1}). \quad (10)$$

This follows fast-weight delta rules Schlag et al. (2021). Under within-frame permutation P , $\mathbf{K}' = P\mathbf{K}$, $\mathbf{V}' = P\mathbf{V}$, and the correction is unchanged because $P^\top P = I$. Thus points remain unordered within a frame while frame order enters the recurrence. This intentionally loses exact cross-frame additivity and is not used in our results.

4. Experiments

4.1. Protocol

We center each object and divide XYZ by its maximum absolute coordinate, using 1,024 points. ShapeNetPart has 13,972 training and 2,874 test shapes; ModelNet40 has 9,840 and 2,468. Models train 20 epochs with AdamW, base learning rate 3×10^{-3} , weight decay 10^{-4} , cosine decay, and task batch sizes 16 and 32. Classification uses 0.1 label smoothing. Values are mean $\pm$ population standard deviation for seeds 0,1,2. Density shift samples without replacement proportional to $\exp(2.5\mathbf{x}^\top \mathbf{d})$ for a fixed-seed random unit direction. The compact PointNet++ models and wider PointNet are controlled local baselines; the latter (442,838 parameters) is not exactly parameter-matched to PointProgrammerNet (529,426).

4.2. Accuracy

PointProgrammerNet is 0.51 points lower in point accuracy, 1.49 lower in instance mIoU, and 0.82 lower in category mIoU than lightweight PointNet++, while using 36.1% fewer parameters and no historical-point maximum.

The state-only classifier reaches 86.59% clean accuracy, 3.25 points below lightweight PointNet++ and 1.31 below wider PointNet. Its density-shift drop is 4.12 points. This is the costlier side of the fixed-state bottleneck, but demonstrates useful classification from mergeable states with no point storage.

4.3. Additivity and fixed state

Across 32 objects, 2–32 ordered and shuffled chunks match one-shot construction with maximum relative error below 4.9×10^{-7} . The three states contain 65,280 float32 scalars (255 KiB). This is fixed, but not universally smaller than raw input: XYZ reaches 255 KiB at 21,760 points, so 1,024 raw XYZ points use less memory. The benefit is bounded history and learned moments.

4.4. State operations

Append/merge, removal, and decay match direct constructions with mean relative errors 2.36×10^{-7} , 3.23×10^{-7} , and 1.92×10^{-7} ; maxima stay below 8.9×10^{-7} . With 16 frames and $\rho = .9$, additive state norm reaches 14.57 and decayed norm 7.45. This verifies algebra, not temporal-task accuracy.

4.5. Streaming efficiency

On an RTX 3070 Ti Laptop GPU we time device-resident state construction for four streams, 256 new points per frame, 30 repetitions, and seven trials. Incremental latency remains near 1.5–1.6 ms. Rebuilding is slightly faster through 1,024 points due to fixed overhead. Speedup is 1.54 $\times$ at 2,048, 3.04 $\times$ at 4,096, and 5.54 $\times$ at 8,192 points; state error stays below 7.8×10^{-7} .

4.6. Controlled architecture audit

Five-epoch three-seed screens isolate design choices. Separated radii improve segmentation instance mIoU by 0.51 ± 0.18 points. Unified partition keys improve category mIoU by 0.82 ± 0.36 over mixed keys. Middle-reference residual classification improves clean and shifted accuracy by 2.67 ± 1.60 and 3.78 ± 1.71 over concatenation. The 25 $\times$ middle-radius rate has variable clean change (0.13 ± 1.03) but consistent shifted gain (0.91 ± 0.33), so we treat it as a robustness-oriented optimization choice.

5. Discussion and Limitations

The practical gain is operational: encode a frame, merge its state, and release the points; encode chunks on separate workers; retain only segmentation query coordinates; or update a classifier from a new evidence chunk. No history-wide discrete comparison is used.

The same constraint is an information bottleneck. PointNet++ forms input-centered neighborhoods and PointNet preserves channelwise extremes; PointProgrammerNet uses finite moment tables. The offline gap is therefore expected, especially for classification. State memory is larger than bare XYZ for small clouds, and segmentation still requires one read per requested coordinate.

Normals, color, or intensity can enter values while XYZ keys space. Normals might also define a product key, subject to orientation and bandwidth. Current evaluation covers one dataset per task, XYZ input, one timing

Table 1. ShapeNetPart segmentation with XYZ input and 1,024 points. PointProgrammerNet performs no decoded-point global pooling. Values are mean $\pm$ population standard deviation over three seeds.

Model	Point acc. (%)	Ins. mIoU (%)	Cat. mIoU (%)	Params
Lightweight PointNet++	93.41 $\pm$ 0.05	83.88 $\pm$ 0.11	79.51 $\pm$ 0.24	573,106
PointProgrammerNet	92.90 $\pm$ 0.01	82.39 $\pm$ 0.12	78.68 $\pm$ 0.07	366,389

Table 2. ModelNet40 classification with 1,024 points. Density denotes evaluation under the fixed density-gradient shift. PointProgrammerNet retains only fixed-size FWP states and uses no global max.

Model	Clean (%)	Density (%)	Drop (pp)	Params
Lightweight PointNet++	89.84 $\pm$ 0.43	85.47 $\pm$ 0.88	4.38 $\pm$ 1.18	207,784
Wider PointNet	87.90 $\pm$ 0.27	87.55 $\pm$ 0.38	0.35 $\pm$ 0.12	442,838
PointProgrammerNet	86.59 $\pm$ 0.23	82.47 $\pm$ 0.93	4.12 $\pm$ 0.71	529,426

GPU, and local controlled baselines. We have not measured unseen-query accuracy, temporal calibration, per-point class stability, or delta adaptation. Continuous logits do not imply monotonic improvement or a continuous hard label.

6. Conclusion

PointProgrammerNet treats a point cloud as a program for fixed additive matrices. Continuous keys and second-order values permit unordered writes, merging, removal, decay, and history-independent updates. Coordinate segmentation and state-only classification avoid decoded-point global maximum. Although offline accuracy remains below hierarchical baselines, the model retains useful accuracy under a stronger streaming constraint and provides a basis for frame-level delta transitions.

References

- Chang, A.X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., Xiao, J., Yi, L., Yu, F., 2015. Shapenet: An information-rich 3d model repository. arXiv:1512.03012 .
- Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F., 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention, in: ICML, pp. 5156–5165.
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., Teh, Y.W., 2019. Set transformer: A framework for attention-based permutation-invariant neural networks, in: ICML, pp. 3744–3753.
- Qi, C.R., Su, H., Mo, K., Guibas, L.J., 2017a. Pointnet: Deep learning on point sets for 3d classification and segmentation, in: CVPR, pp. 652–660.
- Qi, C.R., Yi, L., Su, H., Guibas, L.J., 2017b. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: NeurIPS.
- Schlag, I., Irie, K., Schmidhuber, J., 2021. Linear transformers are secretly fast weight programmers, in: ICML, pp. 9355–9366.
- Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J., 2015. 3d shapenets: A deep representation for volumetric shapes, in: CVPR, pp. 1912–1920.
- Yi, L., Kim, V.G., Ceylan, D., Shen, I.C., Yan, M., Su, H., Lu, C., Huang, Q., Sheffer, A., Guibas, L., 2016. A scalable active framework for region annotation in 3d shape collections. ACM Transactions on Graphics 35, 210:1–210:12.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A.J., 2017. Deep sets, in: NeurIPS.

Appendix A. Reproducibility

Final identifiers are fwp_dual_read_partition_multiscale_seg and pointprogrammer_state_multiscale_residual_main_r25; parameter counts are 366,389 and 529,426. Both expose build_states and merge_states; task APIs are predict_from_states and classify_states.

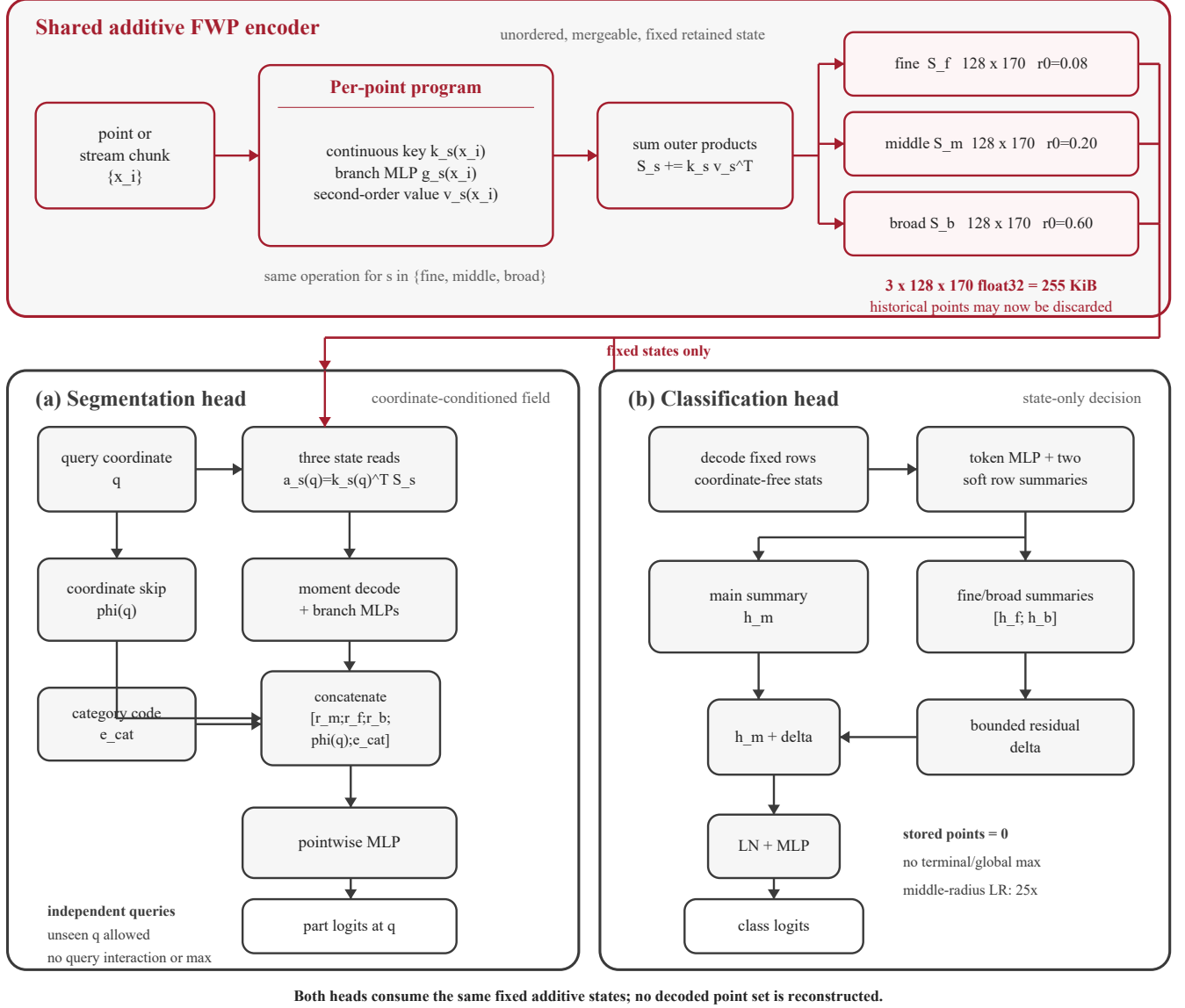


Fig. 1. PointProgrammerNet system architecture. The shared encoder continuously writes each unordered observation into three equal-size, additive FWP states; the historical point list can then be discarded. The segmentation head reads the states only at a requested coordinate and combines the three contextual reads with an information-preserving coordinate skip, which is U-Net-like in function but does not copy a matched-resolution feature map. The classification head operates only on fixed state rows and adds a bounded fine/broad residual to the middle-branch summary. Neither head applies a maximum over decoded points. Orthogonal connectors distinguish the state flow from the two task-specific readouts.

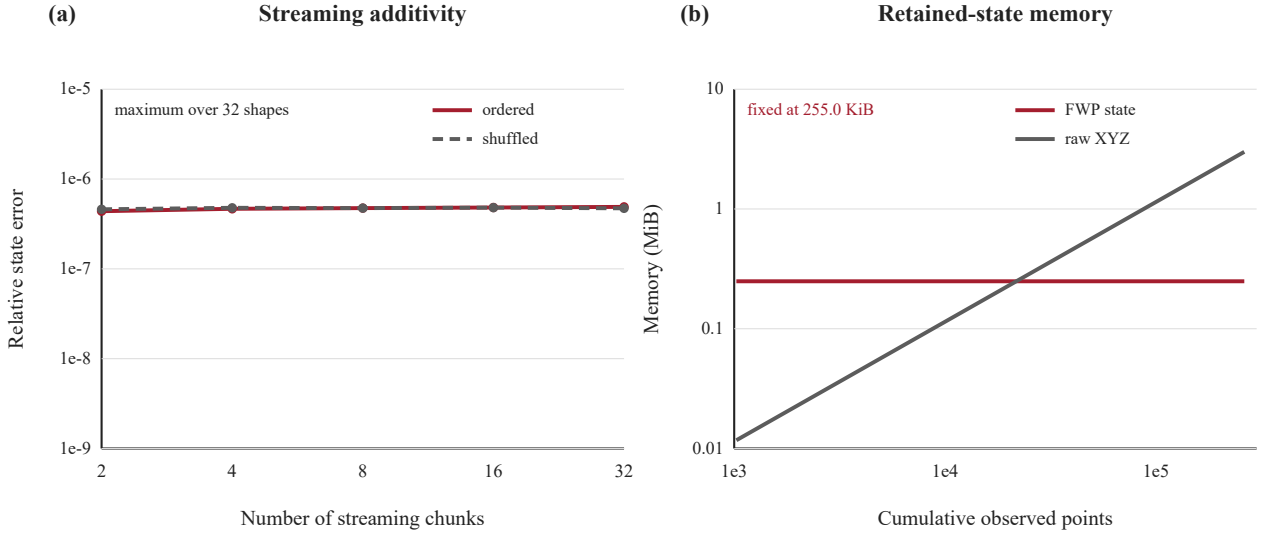


Fig. 2. Additive streaming and fixed retained memory of the final no-max PointProgrammerNet. Ordered and shuffled chunk writes reproduce the single-batch FWP state up to floating-point accumulation error. The three 128×170 float32 states retain 255 KiB independent of stream length, whereas retaining raw XYZ coordinates grows linearly with the number of observations.

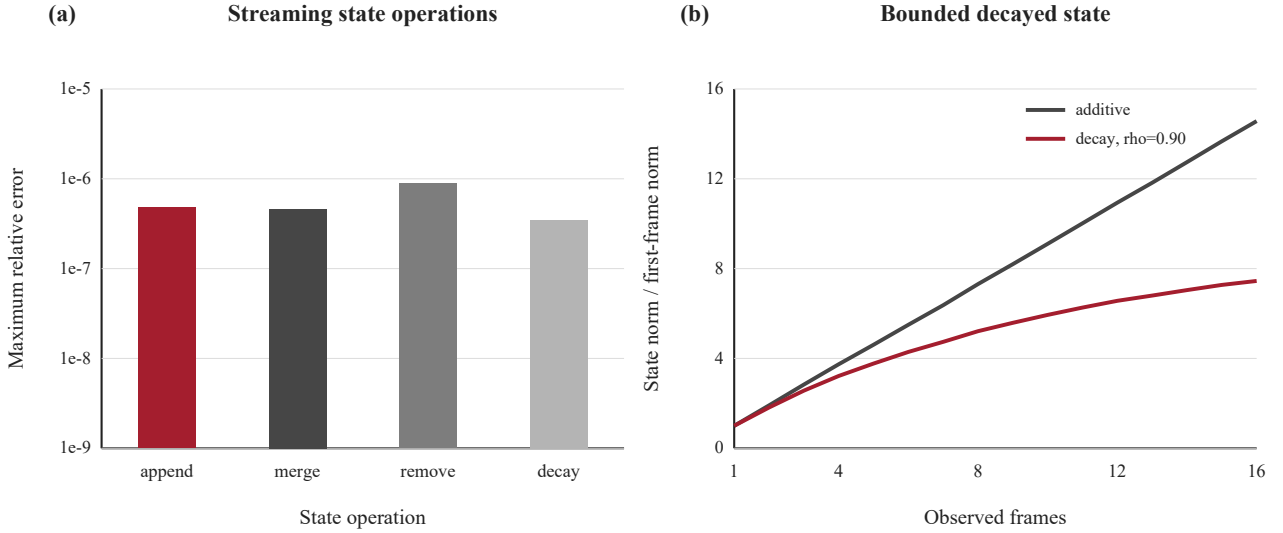


Fig. 3. Native streaming state operations in PointProgrammerNet. (a) Maximum relative errors of append/merge, frame removal, and exponential-decay updates against independently reconstructed offline FWP states over 32 ShapeNetPart objects. (b) Ordinary addition accumulates state magnitude with the number of frames, while exponential decay bounds historical influence without changing the state dimensions. This experiment verifies state algebra rather than downstream temporal-task accuracy.

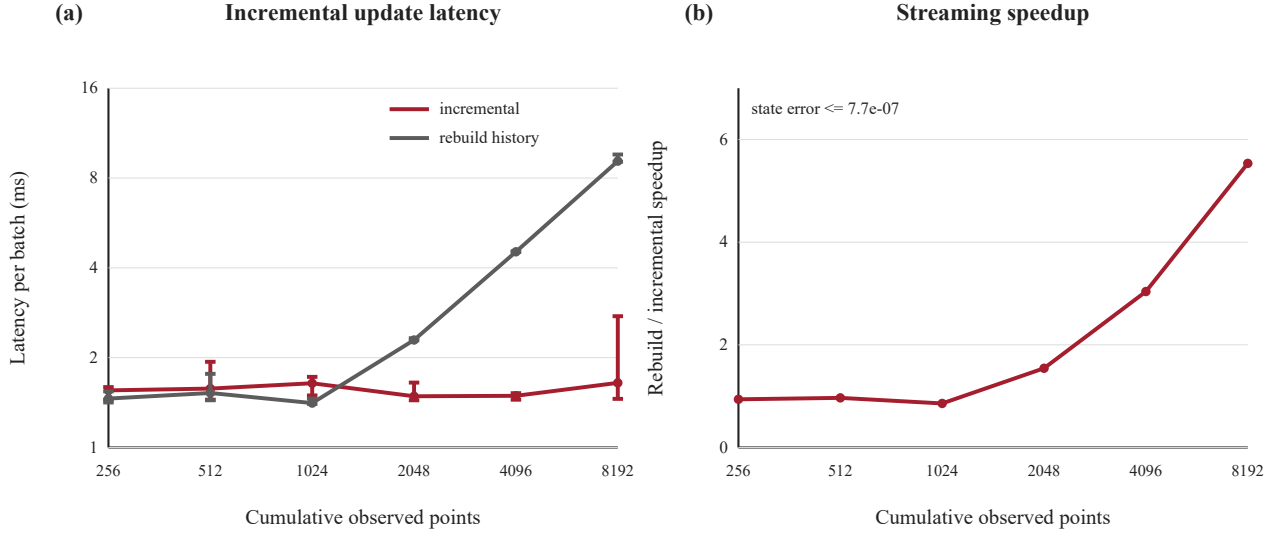


Fig. 4. Measured FWP state-construction cost for PointProgrammerNet on device-resident point streams. (a) An incremental update processes only the new 256-point frame, while offline reconstruction repeatedly processes all accumulated points. Values are median wall-clock latency over seven interleaved trials, each containing 30 repetitions, for a batch of four streams; whiskers span the observed trial range. (b) The resulting speedup increases with history length; incremental and rebuilt states remain numerically equivalent.

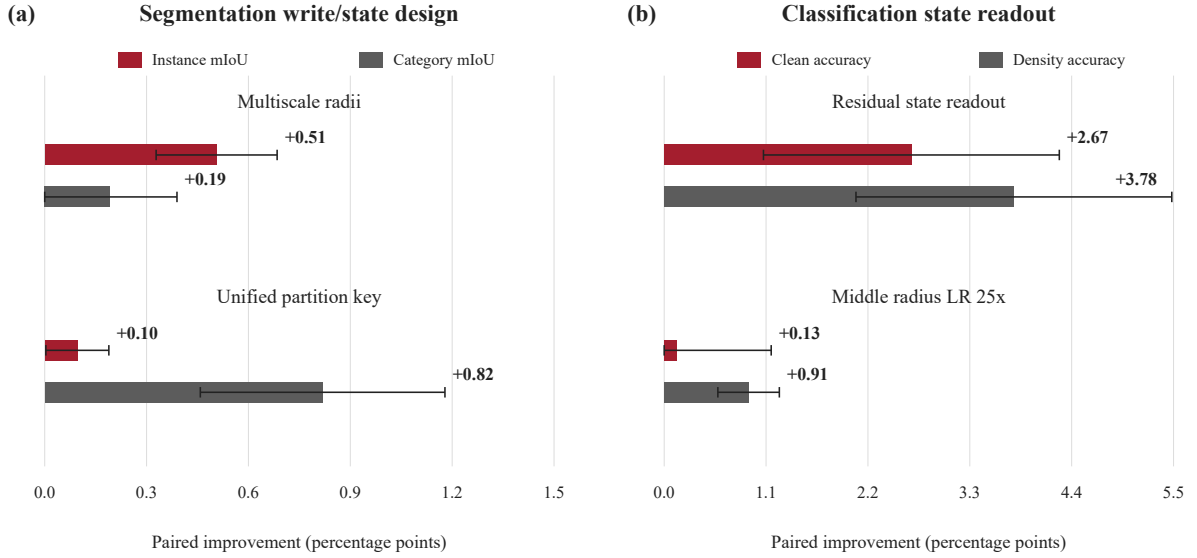


Fig. 5. Three-seed screening ablations supporting the frozen PointProgrammerNet design. (a) For segmentation, separated initial radii improve equal partition states, while a unified partition key improves the multiscale mixed-key control. (b) For state-only classification, a middle-reference residual readout improves both clean and density-shift accuracy over direct concatenation; accelerating only the middle radius further improves density-shift accuracy. Bars show paired mean changes and whiskers show the population standard deviation. These five-epoch results select components and are not the final benchmark.